

Transition: DS + Algorithms

Runtime
Analysis



Last time: Big- $\mathcal{O}$ Proofs

Worksheet review:

Recap Let f and g be functions from $\mathbb{Z}^+$ to $\mathbb{R}^+$ $\rightarrow$ counting things
why?

We say $f(n) = O(g(n))$ if $\exists$
constants c and n_0 s.t. $\forall n > n_0,$
 $\underline{f(n)} \leq \underline{c \cdot g(n)}$

Pick a worksheet one:

That limit definition

I did $f(n) = O(g(n))$ if $\exists n_0 \ \& \ c$ s.t.

$$\forall n > n_0, \quad f(n) \leq c \cdot g(n)$$

Take that inequality:

So $f(n) = O(g(n)) \Leftrightarrow$

Big Ω

Let f and g be functions from $\mathbb{Z}^+ \rightarrow \mathbb{R}^{\geq}$

We say f is $\Omega(g(n))$ if

$\exists$ positive constants c and n_0
such that $\forall n > n_0$,

$$f(n) \geq c \cdot g(n)$$

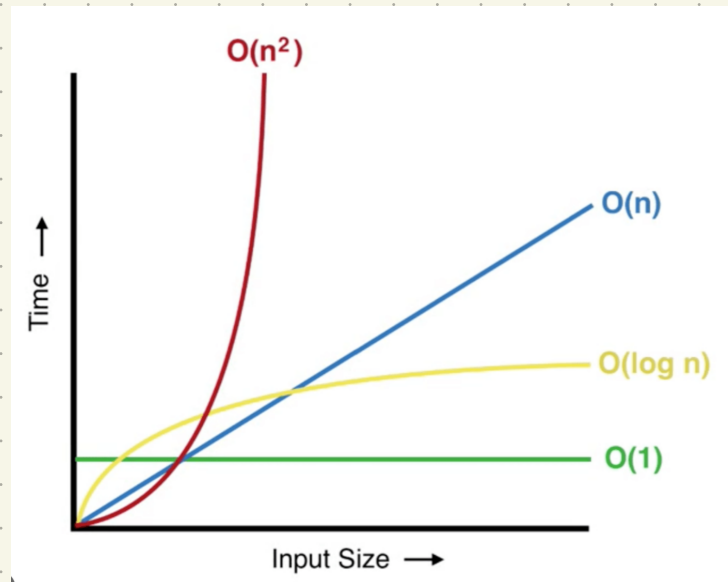
" f is big-omega of g "

Example: $5 \cdot n$ is $\Omega(n)$:

Big- Θ

If f is $O(g)$ and $\Omega(g)$ then f is $\Theta(g)$.

This gives a way to nicely "bucket"
function size:



Theorems

Let $p(n)$ be a degree k polynomial, so

$$p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

for constants a_i and $a_k \neq 0$.

Then $p(n) = \Theta(n^k)$.

Proof:

Big- Θ :

Big-O:

Using this theorem;

Give big- Θ estimates for:

- $f(x) = \frac{1}{8} x^5 + x^3 + 2$

- $f(x) = 20000 x^2 - 100000 x$

- $f(x) = \frac{2x^3}{3000} - x + 2$

Other tricks:

Sometimes $f(n) = \sum_{i=1}^n \log_2 n$

How to get Θ -bound?

Another: $\log_2 n$ versus $\log_{10} n$.

What is a log?

Identities:

Another:

$$\sum_{i=1}^n \log_2 i =$$

Next time:

Analyzing algorithms with $\log-O$.

Reading: recaps the definitions

→ then uses $\log-O$

Example: Function with input $A[1..n]$

$sum \leftarrow 0$

For $i \leftarrow 1$ to n

$sum \leftarrow sum + A[i]$

return sum

Runtime: Count operations!
Can't count directly - depends on
A, & how large.

So, use a function:

sum $\leftarrow$ 0

For $i \leftarrow 1$ to n

sum $\leftarrow$ sum + $A[i]$

return sum

End result: